

课程笔记

FlashAttention: 为什么它能省显存、还更快

从 128K 窗口的 1TB 显存灾难，到分块计算与在线 Softmax

lecture-to-notes 自动讲义

2026 年 6 月 21 日



视频作者/频道: 向量隐修会

发布平台: Bilibili

视频时长: 约 34 分钟

视频链接: <https://www.bilibili.com/video/BV1irPTzGEhG>

目录

1 引言：标准注意力的显存灾难	2
1.1 本章小结	2
2 问题：注意力分数矩阵 S 大到 1TB	2
2.1 改进的方向	2
2.2 本章小结	3
3 思路一：分块计算 (Tiling)	3
4 思路二：在线 Softmax (Online Softmax)	3
4.1 为什么 Softmax 要减最大值	3
4.2 分块带来的难题：各段的 M 不一样	3
4.3 解法：用缩放系数 rescale	3
5 在 GPU 上的执行	4
5.1 标准注意力 vs. FlashAttention 的数据流	6
6 为什么不仅省显存，还更快	6
7 总结与延伸	7
7.1 核心三招	7
7.2 笔者的延伸：放进系列看	7
7.3 拓展阅读	7

1 引言：标准注意力的显存灾难

上一讲我们看清了标准注意力是怎么在 GPU 上一步步算出来的。但那套朴素算法有一个致命问题：对显存的占用极其巨大。本讲的 **FlashAttention** 就是为了解决它——而且解决之后，不仅省显存，还更快。

1.1 本章小结

先量化这个”显存灾难”有多严重，再看 FlashAttention 用什么思路把它化解。

2 问题：注意力分数矩阵 S 大到 1 TB

注意力公式第一步 $S = QK^T$ 会得到一个注意力分数矩阵。它有多大？算一笔账：假设模型窗口 128K、嵌入维度 4096、每层 32 个注意力头。



图 1: 单个注意力头的分数矩阵 S 是 $128K \times 128K$ 。其中 $128K = 128 \times 1024 = 131072$ ，按 FP16（每数 2 字节）计算： $131072 \times 131072 \times 2 / 1024^3 \approx 32 G$ 。²

S 矩阵的存储随窗口长度平方增长

单头就要 32 G，而每层有 32 个头： $32 G \times 32 = 1024 G = 1 TB$ 。每一层注意力都需要 1 TB 显存来存这个分数矩阵——任何显卡都不可能有这么大的显存。更糟的是：输入长度是 128K，而 S 的大小是 $128K \times 128K$ ——它按上下文窗口长度的平方增长。标准注意力面对稍长一点的窗口就完全无能为力。

2.1 改进的方向

很自然的想法是：如果中间不一次性存下这么大的 S 矩阵，而是把它拆小、化整为零，是不是就能绕过这个显存瓶颈？这正是 FlashAttention 要做的事。

²视频画面时间区间：00:01:55–00:02:32。

2.2 本章小结

标准注意力的瓶颈不在算力，而在要把巨大的 S 矩阵整个存下来。FlashAttention 的目标：永远不完整地存储 S 。

3 思路一：分块计算 (Tiling)

既然完整的 S 存不下，那就把 K 、 V 切成多段，一段一段地算——先算第一段、再算第二段，每一次只需要很小的空间，自然就不需要那么大的显存了。

先重排公式：把除法推迟到最后

注意力是 $\text{softmax}(S) V = \frac{e^S}{\sum e^S} V$ 。直接算需要先得到完整的分母 $\sum e^S$ （要看到所有元素）。FlashAttention 把它拆成“分子 $e^S V$ ”和“分母 $\sum e^S$ ”分别累加，最后再相除：

$$\text{输出} = \frac{(e^{S_a} V_a + e^{S_b} V_b + \dots)}{(\sum e^{S_a} + \sum e^{S_b} + \dots)}.$$

分子、分母都可以分段累加；除法放到最后做（这也更适合 GPU）。于是每算完一段，中间结果用完即扔、不必写回，显存占用被压到一个小块的量级。

4 思路二：在线 Softmax (Online Softmax)

分块带来一个新麻烦，根源在于 Softmax 的数值稳定技巧。

4.1 为什么 Softmax 要减最大值

实际算的不是 e^{x_i} 而是 $e^{x_i - M}$ ，其中 M 是这一行的最大值。原因是防止数值溢出：若某个 x_i 很大（如 100）， e^{100} 会大到浮点数存不下。减去最大值后，指数最大为 $e^0 = 1$ ，不会溢出；个别变成极小（如 $e^{-50} \approx 0$ ）也可接受。

4.2 分块带来的难题：各段的 M 不一样

但是——分块后，处理第一段时看不到第二段，只能把 M 当作第一段的最大值（比如 12）；等第二段来了才发现真正的最大值是 18。两段用了不同的 M ，它们的指数结果底数不同、不能直接相加。

4.3 解法：用缩放系数 rescale

关键在于指数运算的一个特性：

$$e^{x-18} = e^{(x-12)-6} = e^{x-12} \cdot e^{-6}.$$

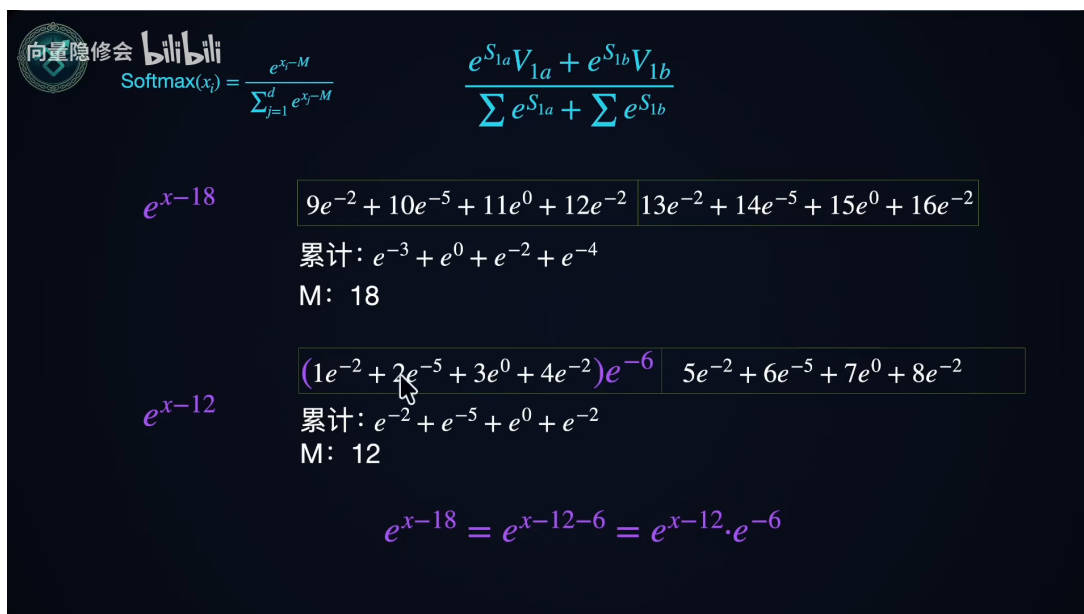


图 2: 在线 Softmax 的核心: 第一段按 $M_1 = 12$ 算出了部分结果, 第二段发现更大的 $M_2 = 18$ 。只需给第一段已经算好的分子与分母乘上缩放系数 $e^{-6} = e^{M_1 - M_2}$, 就等价于“一开始就用 $M_2 = 18$ 算”, 于是两段可以合并。⁴

在线 Softmax 的运行方式

逐段处理; 每段记录该段最大值 m_i 与部分和。一旦新段出现更大的最大值, 就用

$$\text{缩放系数} = e^{m_{\text{旧}} - m_{\text{新}}}, \quad m = \max(m_2 - m_1, 0)$$

把之前累积的分子、分母统一缩放到新的基准上, 再把新段累加进来。这样无需保留任何原始中间数据, 只靠几个标量 (当前最大值、当前分子、当前分母) 就能“边走边合并”——这就是 **Online Softmax**。

5 在 GPU 上的执行

把分块 + 在线 Softmax 落到硬件上。HBM 里准备好两套 (两头) QKV, 由 L2 Cache 把数据读进 SM。

⁴视频画面时间区间: 00:19:57–00:21:30。

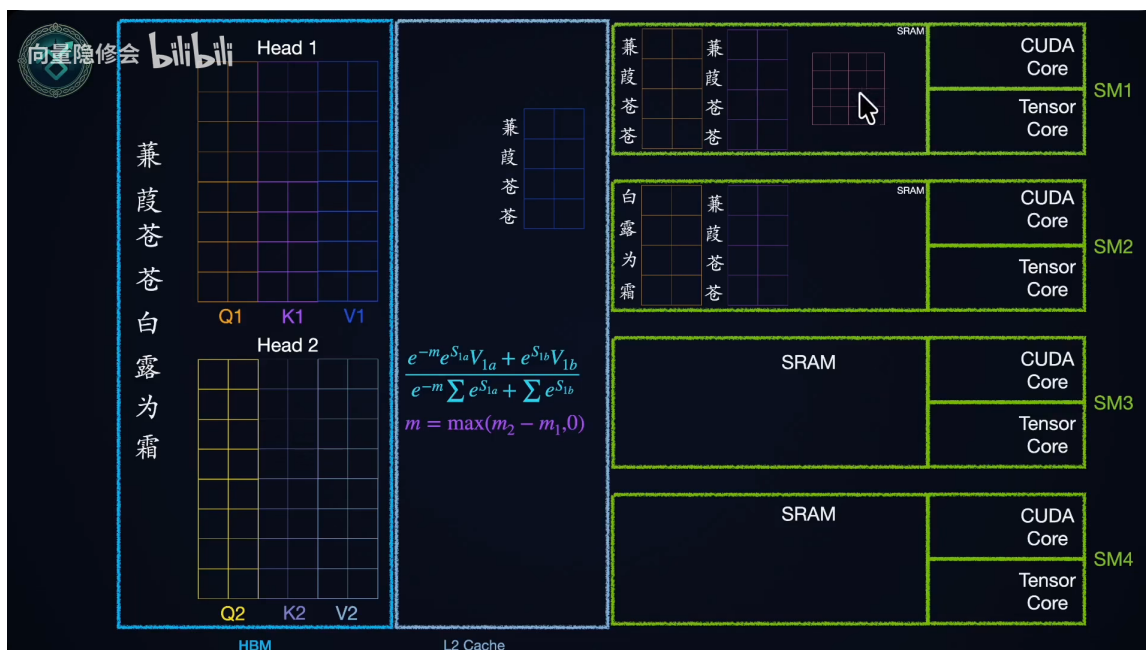


图 3: FlashAttention 的 GPU 执行: 4 个 SM, 前两个处理第一头、后两个处理第二头; 同一头内 SM1 管前四个字、SM2 管后四个字。每个 SM 在自己的 SRAM 里分段算出小的 4×4 得分块、做在线 Softmax (公式中的 $m = \max(m_2 - m_1, 0)$ 即缩放), 算完即扔。⁶

每段的流程是: 把 Q 加载进 SM (直到算完都不释放) $\rightarrow$ 读入第一段 K , 算出小的 4×4 得分块 $\rightarrow$ 记录每行最大值 M 与累计和、做指数 $\rightarrow$ 读入第一段 V , 算 $e^S V \rightarrow$ 把中间得分块直接扔掉 $\rightarrow$ 读第二段 K, V , 重复, 并按需对前一段结果做 rescale, 再相加。

和标准注意力最大的不同

标准注意力会把那个巨大的注意力分数矩阵写回缓存、最终存进 HBM; 而 FlashAttention 里, 那些中间过程零零碎碎地在 SM 内部就被处理掉、用完马上扔, 从不完整地落回 HBM。最终只把每个字的输出结果写回去。

⁶ 视频画面时间区间: 00:24:17-00:24:35。

5.1 标准注意力 vs. FlashAttention 的数据流

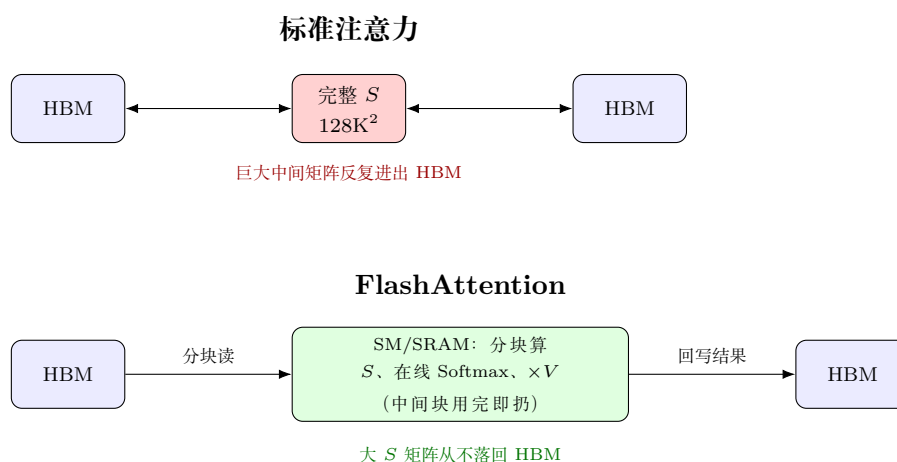


图 4: 两种算法的数据流对比 (笔记自绘): 标准注意力把 $128K^2$ 的中间矩阵反复写进/读出 HBM; FlashAttention 把分块计算全部留在 SM 内的 SRAM 完成, 中间矩阵从不完整落回 HBM, 只回写最终结果。

6 为什么不仅省显存, 还更快

计算时间 + IO 时间

一次计算的总耗时 = 计算时间 + IO (数据搬运) 时间。上一讲的“内存墙”说明: HBM 读写远慢于 SM 内的计算, SM 常常在等数据。FlashAttention 通过**不把中间大矩阵搬进搬出 HBM**, 大幅削减了 IO 时间——于是**既省显存、又更快**。这正是它被称为“*IO-aware* (*IO 感知*)”注意力的原因。



图 5: 最终只把每个字的输出结果写回, 中间分数矩阵全程未落 HBM。一次计算的总时间 = 计算时间 + IO 时间, FlashAttention 主要削减的是后者。⁸

它算的是”精确”注意力，不是近似

值得强调：FlashAttention 分块 + 在线 Softmax 合并出来的结果，与一次性用完整 K, V 算出来的 **完全一致**——它是精确注意力，只是换了一种”省 IO”的计算顺序，没有任何近似或精度损失。

7 总结与延伸

7.1 核心三招

1. **分块 (Tiling)**: 把 K, V 切段，逐段算，中间块用完即扔，避免存下 $128K^2$ 的 S ;
2. **公式重排**: 分子 $e^S V$ 、分母 $\sum e^S$ 分别累加，除法推迟到最后;
3. **在线 Softmax**: 用缩放系数 $e^{m_{\text{旧}} - m_{\text{新}}}$ 处理”各段最大值 M 不同”的问题，边走边合并，无需保留原始中间数据。

7.2 笔者的延伸：放进系列看

- **直接回应上一讲的”内存墙”**: 上一讲指出 SM 常卡在等 HBM 搬数据；FlashAttention 正是那个问题的标准答案——把中间矩阵留在 *SRAM*、不落 HBM，从而省 IO 提速。
- **与 KV Cache 正交**: KV Cache 省的是自回归生成中跨步的重复计算；FlashAttention 省的是单次注意力内部的显存 *IO*。两者解决不同维度的问题，可叠加。
- **”减最大值”的伏笔收回**: 上一讲特意把”为什么 Softmax 要减最大值”留到本讲——因为正是这个 M 在分块后变得”不全局可见”，催生了 online softmax 的 rescale 技巧。
- **一句话记忆**: 大矩阵不落盘，分块算、在线并；省的是 *IO*，换来的是又省显存又快。

7.3 拓展阅读

- Dao et al., *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*, NeurIPS 2022。
- 关键词: *Tiling*、*Online Softmax*、*IO-Aware Attention*、*Safe Softmax*。
- 视频原址: <https://www.bilibili.com/video/BV1irPTzGEhG> (”向量隐修会”注意力优化系列)。

⁸视频画面时间区间: 00:31:00–00:31:30。